

Nyx Haile

Systems & Research Engineer

xyn@mit.edu | LinkedIn | GitHub | Open to relocation

EDUCATION

Massachusetts Institute of Technology

2026

B.S. Computer Science, B.S. Mathematics, Minor in Economics

Cambridge, MA

- Coursework: algorithms, theory of computation, computer systems security, cryptography, statistics.
- Computer Systems Security (6.566) final project: a web server with WebAuthn login, script-injection protection, and payment-style secure transfers; about 2,000 lines.

EXPERIENCE

Partial

2025 – Present

Cofounder & CEO

London

- Designed a perpetual prediction-market exchange where positions are probability distributions over joint price paths across correlated assets, not binary outcomes.
- Specified eleven market types, among them price paths, derived statistics (correlations, lead-lag) and reported statistics, under a single mechanism-design framework.
- Restored strict 0.02 forward-KL / 0.02 relative-price-error projection thresholds as the engine's fail-closed runtime default, keeping looser bounds for diagnostics only; 12 open model failures remain tracked.

Vana

Dec 2022 – July 2023

AI Developer

Remote

- Engineered and iterated prompt templates for personalised AI-generated sticker portraits, tuning style and expression cues for identity retention and consistency.

SELECTED PROJECTS

Repositories marked private are available on request.

Strand (private) | Rust, FoundationDB

Capability-authenticated issue tracker

- Built a federable, capability-authenticated issue tracker in Rust on FoundationDB, with line-level blame and content-addressed rationale.
- Shipped a 16-crate, ~71k-line Rust workspace through v1.0.0 GA (10/10 verification gates) and v1.1.0, then a v2.0 governed-execution prerelease.
- Implemented macaroon capability auth (HMAC-SHA256) with tenant, op-class, expiry and prefix caveats; the server re-derives the writing actor instead of trusting clients.
- Wrote stable-Rust fuzz harnesses for three frozen v1 formats (wire frames, CBOR citations, backup files); the first pass ran 284,846 iterations with zero crashes.
- Bounded FoundationDB transactions with timeouts and retry limits so storage outages surface as typed errors instead of hangs.

Forge (private) | Rust, systemd, RPC

Local-first agent control plane

- Built a Rust daemon, operator CLI, Claude Code hooks and provider proxy that coordinate coding-agent work across multiple git repositories.
- Implemented an atomic file-write tool that records attribution for every write and reverts the file if recording fails; the default recorder fails closed.
- Added bounded, jittered exponential backoff to scheduler dispatch, seeded per task so retry waves de-correlate in production and replay exactly in tests.
- Ran the daemon under systemd with NoNewPrivileges, ProtectSystem=strict and a read-only home, confining writes to its own state directories.

howtowin.lol | Python, PyTorch, CUDA, Redis

World model for post-game teaching

- Designed a Dreamer-lineage RSSM world model for League of Legends combining static context, player priors and event sequences with outcome, next-event, next-frame and next-decision heads.
- Built a GPU kNN retrieval check over ~1.16M anchor states x 544 dims to test whether world-model states cluster by outcome, against random, static and hand-crafted baselines.

MIT Pokerbots | Python

Heads-up poker-variant bot, January 2026

- Built a Python bot for MIT Pokerbots' heads-up poker variant, largely with AI-assisted coding; the team placed about 15th of 70.
- Implemented Monte Carlo equity estimation, with exact enumeration for discards, using quick and deep sampling paths to fit a per-decision compute budget.
- Built a multi-policy architecture (tight, bluff, desperate, lock-win) with a strategic overseer choosing among threshold policies.

- Adopted win-locking after reviewing scrimmage games – fold every remaining hand once the lead guarantees a positive finish – gaining at least five scrimmage places.
- Held top-10 positions on the scrimmage leaderboard in week 2.

Neuro (*private*) | Python, NumPy

Exploratory adaptive sparse neural core

- Implemented temperature-controlled sampled activation, copula-driven rewiring, a loss-plus-sparsity objective and automatic pruning with edge rebalancing.
- Auto-scaled the graph to $n \log n$ nodes in input size with degree capped at $\log(\text{graph size})$.
- Trained online on 60,000 MNIST samples with 4,431 auto-sized hidden nodes, reaching 84.2% running accuracy in one epoch (not held-out test accuracy).
- Wrote a research note relating the model to resource-bounded, stream-based approximations of Solomonoff induction, and where it falls short of them.

Sherlock | Rust, Dolt

Token-usage forensics for agent CLIs

- Built a Rust CLI that ingests agent-CLI logs (Claude Code, Codex, Copilot), attributes token usage per session, and reports what burns tokens without adding value.
- Stored ingested usage in a Dolt versioned-SQL repository.
- Added LLM-narrated before/after cost analysis over Anthropic, OpenAI, Gemini or local backends, and shipped a Claude Code skill that runs it on request.

Conch | Rust, ONNX Runtime

Voice-interview-to-brief CLI

- Built a Rust voice-interview CLI that runs spoken Q&A over mic and TTS at three model depths and writes a persisted brief.
- Integrated streaming 16 kHz audio with cloud or offline on-CPU speech-to-text (Nemotron 0.6B int8 via ONNX Runtime) and a live Ratatui waveform view.

Grimoire (*private*) | Rust, SQLite

Durable agent memory library

- Built a Rust library for durable agent memory on an append-only, SHA-256 hash-chained log with a rebuildable SQLite projection.

Astraia (*private*) | TypeScript, Python, SQLite

Personal multi-agent operations system (on OpenClaw)

- Built a localhost-only operations dashboard (Bun, React, SQLite) where outbound email and messages pass expiring, hash-pinned previews and fail closed.

TECHNICAL SKILLS

Languages: Rust, Python, TypeScript, SQL **ML:** PyTorch, NumPy, CUDA, GPU training and retrieval **Systems:** Linux/systemd, Docker, FoundationDB, PostgreSQL, SQLite, Redis **Web:** SvelteKit, React, Next.js **Hardware:** Fusion 360, Arduino